

SÉCURITÉ DES SYSTÈMES

M1 CSIA (S2)

M. MEDEDJEL

2025-2026

COURS 5 (CHAPITRE III):

VULNÉRABILITÉ BINAIRE (*BUFFER OVERFLOW*)

INTRODUCTION

La vulnérabilité binaire est une faille de sécurité relative aux fichiers binaires exécutables.

Ce type de vulnérabilités peut être exploité par des attaquants pour compromettre la sécurité d'un système ou d'une application.

Un cas typique de cette vulnérabilité est le débordement du tampon mémoire ou ce que l'on appelle communément le « ***Buffer overflow*** ».

INTRODUCTION

Le *Buffer overflow*, est l'une des vulnérabilités les plus courantes qui résulte d'une programmation imprudente dans les applications.

Elle est connue depuis qu'elle a été utilisée pour la première fois par le ver Internet **Morris en 1988**.

Ce chapitre nous verrons comment un débordement de tampon se produit et quelles méthodes permettent de l'empêcher ou de le détecter.

OBJECTIFS

Les objectifs de ce cours peuvent être résumés dans ce qui suit :

- ☐ Définir ce qu'est un débordement de tampon et énumérer les principales conséquences.
- ☐ Expliquer le fonctionnement d'un débordement de tampon sur la pile.
- ☐ Définir le shellcode et décrire son utilisation dans une attaque par débordement de tampon.
- ☐ Présenter les principales défenses contre ce type d'attaque.

NOTIONS DE BASE

- Un débordement de tampon se produit lorsqu'un programme écrit des données au-delà des limites d'un tampon de taille fixe.
- Cette écriture dépasse la zone prévue et écrase des emplacements mémoire adjacents.
- Ces emplacements peuvent contenir :
 - d'autres variables ou paramètres,
 - des informations de contrôle du programme (adresses de retour, pointeurs de pile, etc.).

NOTIONS DE BASE

Les conséquences de cette erreur incluent :

- La corruption des données utilisées par le programme,
→ comportement imprévisible du programme.
- Un transfert de contrôle inattendu dans le programme,
→ exécution arbitraire de code.
- D'éventuelles violations d'accès à la mémoire,
→ déclenchement d'exceptions.
- Très probablement, l'arrêt éventuel du programme.
→ crash ou interruption par le SE.

NOTIONS DE BASE

Remarque:

- Dans le cadre d'une attaque, le débordement peut être utilisé pour détourner le flux d'exécution vers du code choisi par l'attaquant.
- Ce code malveillant est alors exécuté avec les privilèges du processus vulnérable, ce qui peut donner un contrôle complet sur le système ciblé.

NOTIONS DE BASE

Exemple :

Pour illustrer le fonctionnement d'un débordement de tampon, on considère la fonction écrite en C présentée sur la figure suivante.

```
#include <stdio.h>
#include <string.h>
int main(int argc, char *argv[]) {
    char buffer[5];
    strcpy(buffer, argv[1]);
    printf("Buffer contents: %s\n", buffer);
    return 0;
}
```

Fig 1.a. Code C d'un débordement de tampon de base

NOTIONS DE BASE

Description de l'exemple :

Le programme lit un argument de ligne de commande (`argv[1]`), le copie dans un tampon buffer, puis affiche ce tampon avec `printf`.

Or le tampon ne fait que 5 octets, ce qui est insuffisant pour la plupart des chaînes et ouvre la porte à un débordement de tampon.

NOTIONS DE BASE

Description de l'exemple :

Si l'argument de ligne de commande, passé au programme, dépasse 4 caractères (+ '\0'), la fonction strcpy écrasera des zones mémoire adjacentes, ce qui peut entraîner :

- Un plantage du programme (*segmentation fault*).
- Une altération des données adjacentes.
- Une **exploitation de sécurité** pour exécuter un code malveillant (injection de code).

NOTIONS DE BASE

Étapes pour exploiter un débordement de tampon:

- 1) **Identifier** un programme vulnérable où un dépassement de tampon peut être déclenché via des données externes contrôlées par l'attaquant (entrée utilisateur, fichier, trame réseau, etc.).
- 2) **Comprendre** comment ce tampon est stocké en mémoire afin de repérer les zones adjacentes sensibles et, si possible, modifier le flux d'exécution du programme.

STACK BUFFER OVERFLOW

- Un débordement de tampon de pile (*Stack Buffer Overflow*) se produit lorsque le tampon vulnérable est une variable locale stockée sur la pile d'une fonction. Cette attaque est souvent appelée ***stack smashing***.
- Les piles (stacks) sont utilisées pour:
 - gérer les appels de fonctions et procédures,
 - allouer la mémoire des variables locales et des paramètres,
 - stocker les informations de contrôle, comme l'adresse de retour.

STACK BUFFER OVERFLOW

1. Rappel sur le mécanisme d'appel de fonction

- Quand une fonction appelle une autre, elle doit au minimum enregistrer l'adresse de retour pour que l'exécution revienne au bon endroit une fois la fonction appelée terminée.
- Elle doit aussi réserver de la place pour :
 - les paramètres passés à la fonction appelée,
 - et, éventuellement, certaines valeurs de registres qu'elle souhaite retrouver après l'appel.
- L'ensemble de ces informations est stocké sur la pile dans une structure appelée **cadre de pile** (*stack frame*).

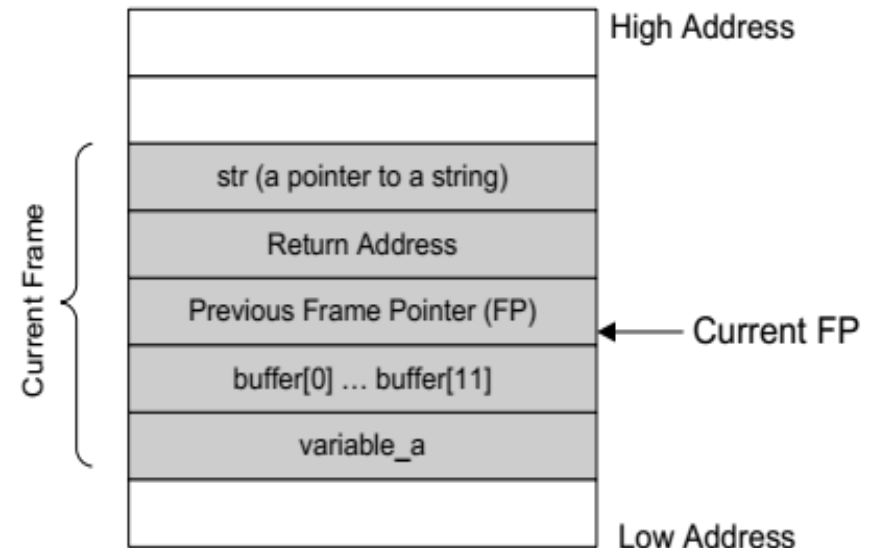
STACK BUFFER OVERFLOW

➤ Cadre de la pile (Stack frame) :

La Figure 2 illustre la structure du cadre de la pile à l'exécution d'une fonction.

```
void func (char *str) {  
    char buffer[12];  
    int variable_a;  
    strcpy (buffer, str);  
}  
  
Int main() {  
    char *str = "I am greater than 12 bytes";  
    func (str);  
}
```

(a) A code example



(b) Active Stack Frame in func()

Fig 2. : Structure du cadre de pile d'une fonction

STACK BUFFER OVERFLOW

Exemple :

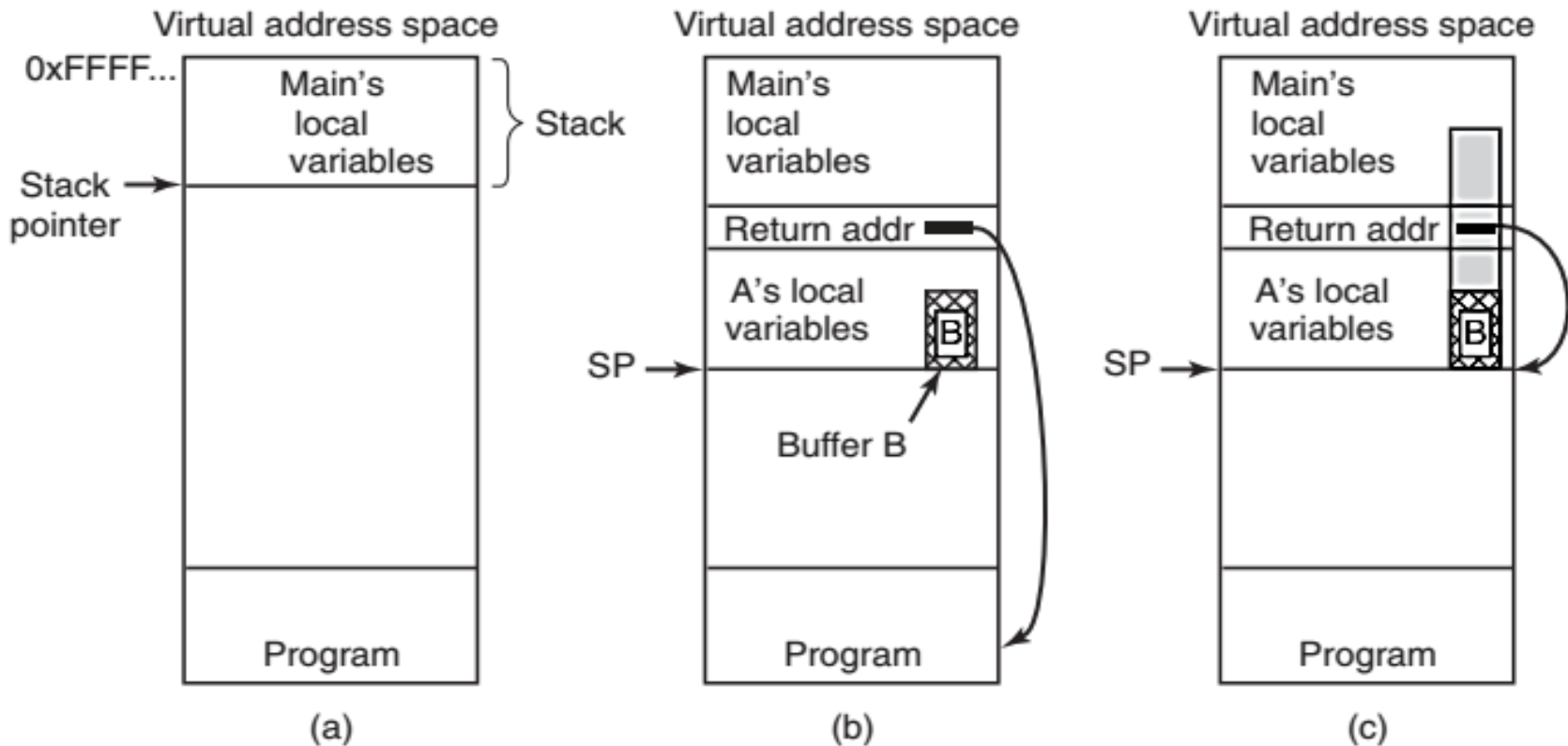


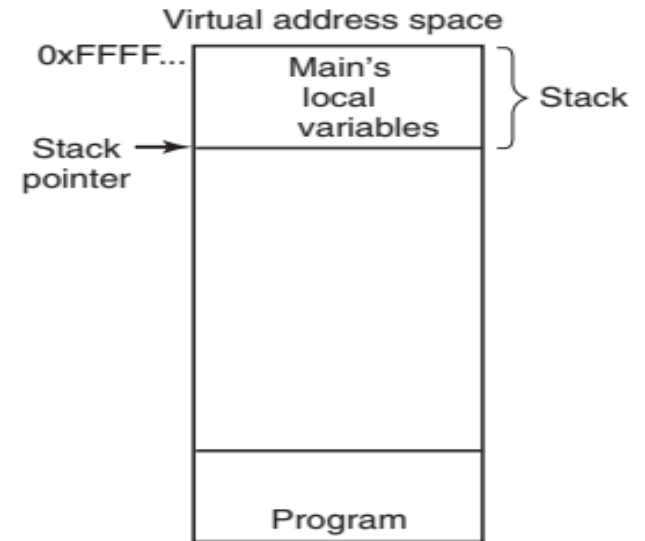
Fig 3. Exemple de cadre de pile : (a) Programme principal en cours d'exécution. (b) Après l'appel de la procédure A. (c) Débordement de tampon.

Source : [2]

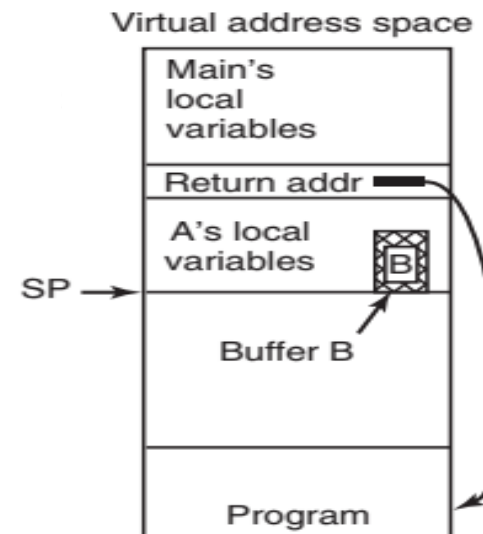
STACK BUFFER OVERFLOW

Illustration : appel de procédure et pile

- Dans la figure 3(a), seul le programme principal s'exécute et ses variables locales sont stockées sur la pile.
- Lorsqu'il appelle la procédure **A** (figure 3(b)) :
 - l'adresse de retour (instruction suivant l'appel) est d'abord empilée,
 - puis le contrôle passe à **A**, qui réserve de l'espace sur la pile pour ses variables locales, comme le tampon **B**.



(a)

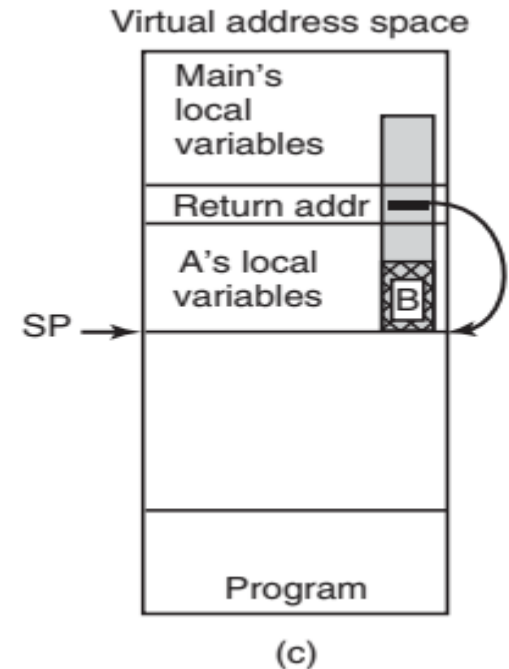


(b)

STACK BUFFER OVERFLOW

Illustration : appel de procédure et pile

- La figure 3(c) montre un débordement : trop d'octets sont copiés dans le tampon **B**, dépassant sa taille.
- Ces données écrasent alors d'autres zones de la pile, notamment l'adresse de retour empilée précédemment, ce qui provoque généralement un blocage immédiat du programme.



STACK BUFFER OVERFLOW

2. Attaque par Buffer Overflow

- Dans un scénario d'attaque, l'attaquant envoie une entrée spécialement construite pour dépasser le **tampon B** et écraser **l'adresse de retour** avec **l'adresse de ce même tampon**.
- Quand la fonction **A** se termine, le programme ne revient plus au code légitime : il saute au début du tampon **B** et exécute les octets qu'il contient comme du code, plaçant ainsi le programme (et potentiellement la machine) sous le contrôle de l'attaquant.
- Le code injecté ouvre souvent un *shell* (par exemple via un appel système `exec`), offrant à l'intrus un accès direct au système compromis.

STACK BUFFER OVERFLOW

3. Code malveillant (Shellcode)

- Le code injecté dans le tampon est appelé **shellcode**.
- Traditionnellement, sa fonction principale est de lancer un interpréteur de commandes (un *shell*), offrant à l'attaquant l'accès à tous les programmes du système avec les privilèges du processus compromis.

STACK BUFFER OVERFLOW

3. Code malveillant (Shellcode)

Exemples de shellcode

- ✓ Sur les systèmes **UNIX**, le shellcode appelle typiquement la fonction système `execve("/bin/sh")`, qui remplace le programme courant par un shell (Bourne, bash, etc.) exécuté avec les privilèges du processus compromis.
- ✓ Sur **Windows**, il invoque en général `system("cmd.exe")` (ou `command.com` sur les versions plus anciennes) pour lancer une invite de commandes avec les mêmes privilèges.

STACK BUFFER OVERFLOW

3. Code malveillant (Shellcode)

Caractéristiques du shellcode

- Le shellcode est un code machine, c'est-à-dire une suite d'instructions binaires qui réalise une action malveillante choisie par l'attaquant.
- Comme il doit s'exécuter et interagir avec le système cible, il est écrit pour une **architecture** processeur et souvent pour un **SE** particuliers.
- Les attaques par débordement de tampon visent donc généralement un logiciel précis dans un environnement bien déterminé.

STACK BUFFER OVERFLOW

➤ Développement de Shellcode :

Pour illustrer la structure de base du shellcode, on peut étudier une attaque classique qui se contente de lancer un shell sur un système Linux. Le shellcode doit alors reproduire la logique présentée dans la **figure 4(a)**, c'est-à-dire exécuter le programme de shell avec les bons paramètres.

```
int main (int argc, char *argv[])
{
    char *sh;
    char *args[2];

    sh = "/bin/sh";
    args[0] = sh;
    args[1] = NULL;
    execve (sh, args, NULL);
}
```

Fig. 4. Exemple d'un Shellcode UNIX
(a) Code C du Shellcode

STACK BUFFER OVERFLOW

➤ Développement de Shellcode :

- Le code prépare d'abord tous les arguments nécessaires à l'appel système `execve()`, y compris une liste minimale d'arguments et de variables d'environnement.
- Il invoque ensuite `execve`, qui remplace le programme courant par le shell.
- Pour obtenir le shellcode, ce code de haut niveau doit être compilé en langage machine, c-à-d en une suite d'octets directement exécutable par le processeur.

```
int main (int argc, char *argv[])
{
    char *sh;
    char *args[2];

    sh = "/bin/sh";
    args[0] = sh;
    args[1] = NULL;
    execve (sh, args, NULL);
}
```

STACK BUFFER OVERFLOW

➤ Développement de Shellcode :

Le résultat du processus de conception du Shellcode est un code similaire à celui illustré à la **Figure 4.b**.

```
    nop
    nop                                //end of nop sled
    jmp find                           //jump to end of code
cont: pop %esi                         //pop address of sh off stack into %esi
    xor %eax, %eax                     //zero contents of EAX
    mov %al, 0x7(%esi)                 //copy zero byte to end of string sh (%esi)
    lea (%esi), %ebx                   //load address of sh (%esi) into %ebx
    mov %ebx, 0x8(%esi)                //save address of sh in args [0] (%esi+8)
    mov %eax, 0xc(%esi)                //copy zero to args[1] (%esi+c)
    mov $0xb, %al                      //copy execve syscall number (11) to AL
    mov %esi, %ebx                     //copy address of sh (%esi) into %ebx
    lea 0x8(%esi), %ecx                //copy address of args (%esi+8) to %ecx
    lea 0xc(%esi), %edx                //copy address of args[1] (%esi+c) to %edx
    int $0x80                          //software interrupt to execute syscall
find: call cont                        //call cont which saves next address on stack
sh:  .string "/bin/sh"                 //string constant
args: .long 0                          //space used for args array
     .long 0                           //args[1] and also NULL for env array
```

Fig. 4. Exemple d'un Shellcode UNIX
(b) Code assembleur x86 équivalent

STACK BUFFER OVERFLOW

➤ Lien entre C, assembleur et shellcode:

- Ce code est écrit en assembleur x86 pour processeurs Intel et implémente la même logique que le programme C de la figure 4(a) (préparation des arguments puis appel de `execve`).
- Après assemblage, on obtient la séquence de code machine illustrée en figure 4(c) : c'est cette suite d'octets qui sera utilisée comme shellcode dans l'attaque.

90	90	eb	1a	5e	31	c0	88	46	07	8d	1e	89	5e	08	89
46	0c	b0	0b	89	f3	8d	4e	08	8d	56	0c	cd	80	e8	e1
ff	ff	ff	2f	62	69	6e	2f	73	68	20	20	20	20	20	20

Fig. 4. Exemple d'un Shellcode UNIX
(c) Valeurs hexadécimales pour le code machine x86 compilé

STACK BUFFER OVERFLOW

Remarque:

Pour faciliter la lecture du code assembleur:

- le **tableau 1** fournit une liste d'instructions x86 les plus courantes,
- et le **tableau 2** récapitule les principaux registres du processeur évoqués dans le code (par exemple EAX, EBX, ECX, ESP, EBP).

STACK BUFFER OVERFLOW

Tableau 1 : Quelques instructions courantes du langage d'assemblage x86

MOV src, dest	copy (move) value from src into dest
LEA src, dest	copy the address (load effective address) of src into dest
ADD / SUB src, dest	add / sub value in src from dest leaving result in dest
AND / OR / XOR src, dest	logical and / or / xor value in src with dest leaving result in dest
CMP val1, val2	compare val1 and val2, setting CPU flags as a result
JMP / JZ / JNZ addr	jump / if zero / if not zero to addr
PUSH src	push the value in src onto the stack
POP dest	pop the value on the top of the stack into dest
CALL addr	call function at addr
LEAVE	clean up stack frame before leaving function
RET	return from function
INT num	software interrupt to access operating system function
NOP	no operation or do nothing instruction

STACK BUFFER OVERFLOW

Tableau 2 : Quelques registres x86

32 bit	16 bit	8 bit (high)	8 bit (low)	Use
%eax	%ax	%ah	%al	Accumulators used for arithmetical and I/O operations and execute interrupt calls
%ebx	%bx	%bh	%bl	Base registers used to access memory, pass system call arguments and return values
%ecx	%cx	%ch	%cl	Counter registers
%edx	%dx	%dh	%dl	Data registers used for arithmetic operations, interrupt calls and IO operations
%ebp				Base Pointer containing the address of the current stack frame
%eip				Instruction Pointer or Program Counter containing the address of the next instruction to be executed
%esi				Source Index register used as a pointer for string or array operations
%esp				Stack Pointer containing the address of the top of stack

CONTRE-MESURES

Défenses contre les attaques de débordement de tampon

Les mécanismes de protection visent soit à empêcher ces attaques, soit au moins à les détecter puis interrompre leur exécution.

On distingue généralement deux familles d'approches :

- **Défenses à la compilation:** elles renforcent les programmes au moment de la compilation pour les rendre plus résistant aux débordements.
- **Défenses à l'exécution:** visent à détecter et annuler les attaques dans les programmes existants.

CONTRE-MESURES

a) Défenses à la compilation

Les défenses au moment de la compilation visent à prévenir ou à détecter les débordements de tampon en instrumentant les programmes au moment de leur compilation.

Principales possibilités :

- Choisir un **langage de haut niveau sûr pour la mémoire**, qui :
 - réalise des vérifications automatiques des limites des tableaux
 - gère automatiquement la mémoire
 - Exp : Python, Java, Rust, C# et Go.
- Activer des **options de compilation de protection de la pile**, qui détectent la corruption du cadre de pile (par exemple ajout de canaris de pile ou protections similaires).

CONTRE-MESURES

a) Défenses à la compilation

▪ StackGuard (*Canary value*)

Insère une petite valeur de contrôle appelée « *canary* » juste avant l'adresse de retour dans le cadre de pile d'une fonction.

Lorsqu'un tampon de pile déborde dans l'adresse de retour de la fonction, cette valeur est forcément écrasée.



CONTRE-MESURES

a) Défenses à la compilation

▪ Fortification de la mémoire (FORTIFY_SOURCE)

Remplace certaines fonctions vulnérables (strcpy, sprintf, etc.) par des versions sécurisées qui **vérifient la taille des tampons**.

Exemple de substitutions :

- strcpy() → __strcpy_chk()
- sprintf() → __sprintf_chk()
- memcpy() → __memcpy_chk()

CONTRE-MESURES

b) Défenses à l'exécution

- Les protections à l'exécution sont particulièrement intéressantes car elles ne nécessitent pas de **recompiler** les applications vulnérables.
- Elles sont généralement déployées via des **mise à jour** du système d'exploitation ou d'outils de sécurité, ce qui permet de protéger des programmes existants sans modifier leur code source.

CONTRE-MESURES

b) Défenses à l'exécution

Les défenses d'exécution les plus courantes incluent :

- Protection de l'espace d'adressage exécutable
- Randomisation de l'espace d'adressage
- Pages de garde

CONTRE-MESURES

b) Défenses à l'exécution

■ Protection de l'espace exécutable:

- Le principe est de marquer certaines zones mémoire, comme la pile ou le tas, comme non exécutables; de sorte qu'une tentative d'exécution de code machine dans ces régions provoquera une exception.
- Elle utilise des fonctionnalités matérielles telles que le bit **NX (Non-eXecutable)**.
- Sous Linux, par exp. l'option **-Wl,-z,noexecstack**, indique à l'éditeur de liens de marquer la pile comme non exécutable.

CONTRE-MESURES

b) Défenses à l'exécution

■ Randomisation de l'espace d'adressage (ASLR):

- Dans une attaque classique par débordement de pile, l'attaquant doit pouvoir **prédire l'adresse** du tampon ciblé afin d'y faire pointer l'adresse de retour et d'exécuter son shellcode.
- L'ASLR (*Address Space Layout Randomization*) rend cette prédiction beaucoup plus difficile en déplaçant de façon aléatoire les zones mémoire (pile, tas, bibliothèques, etc.) à chaque exécution ou pour chaque processus.

CONTRE-MESURES

b) Défenses à l'exécution

■ Pages de garde:

- Cette technique insère des pages de mémoire « vides » entre des zones critiques de l'espace d'adressage d'un processus.
- Ces pages sont signalées dans la MMU comme des adresses illégales, et toute tentative d'y accéder entraîne l'abandon du processus.
- Cela peut être utile pour une application qui doit surveiller la croissance de grandes structures de données dynamiques.

CONTRE-MESURES

Autres techniques avancées

➤ Control Flow Integrity (CFI)

- Cette technique **empêche la redirection du flux d'exécution** vers des adresses non prévues. Elle est utilisée dans les compilateurs modernes (**LLVM, GCC**).
- Le compilateur construit (ou approxime) un graphe de flot de contrôle (CFG) décrivant quelles transitions sont autorisées.
- Pour chaque saut indirect (retour de fonction, pointeur de fonction, appel virtuel), une vérification est insérée afin de bloquer les cibles non autorisées.

CONTRE-MESURES

Autres techniques avancées

➤ Shadow Stack

- Cette technique est une seconde pile qui **duplique** uniquement les adresses de retour des appels de fonctions.
- À chaque instruction d'appel, l'adresse de retour est poussée à la fois sur la pile normale et sur la shadow stack.
- Au moment du retour de fonction, l'adresse de retour lue sur la pile normale est comparée à celle de la shadow stack.

CONCLUSION

- ✓ Le **débordement de tampon** est longtemps resté l'une des principales vulnérabilités logicielles : il se produit lorsqu'un programme écrit plus de données que la taille allouée au tampon, débordant sur la mémoire adjacente.
- ✓ Lorsque le tampon se trouve sur la **pile**, le débordement peut écraser l'adresse de retour, ce qui force le programme à sauter vers la nouvelle adresse écrite par l'attaquant.
- ✓ En plaçant du **code malveillant** à l'emplacement pointé par cette nouvelle adresse de retour, l'attaquant peut faire exécuter ce code par le programme vulnérable.
- ✓ Si ce programme dispose de **privilèges élevés** (Set-UID, service réseau, pilote de périphérique, etc.), le code malveillant s'exécute avec les mêmes droits, pouvant mener à une compromission totale du système.

CONCLUSION

- ✓ Les développeurs doivent appliquer des **pratiques de codage sécurisé** :
 - vérification systématique des limites,
 - contrôle de la taille des données copiées,
 - validation d'entrées et évitement des fonctions dangereuses (strcpy, gets, etc.).
- ✓ De nombreuses contre-mesures existent (pile non exécutable, ASLR, canaris de pile, ...) et sont déjà intégrées aux SE, compilateurs, outils et bibliothèques.

CONCLUSION

- ✓ Ces protections ne sont **pas infailibles** : certaines sont moins efficaces sur les architectures 32 bits ou peuvent être contournées par des techniques avancées.
- ✓ L'attaque « **return-to-libc** » permet de contourner la contre-mesure de pile non exécutable en redirigeant l'exécution vers des fonctions système légitimes déjà chargées en mémoire (par exemple dans libc).

**MERCI POUR VOTRE
ATTENTION.**

RÉFÉRENCES:

- [1] Andrew Tanenbaum « Systèmes d'exploitation » (Ed. 3).
Pearson Education (2008).
- [2] ANDREW S. TANENBAUM & HERBERT BOS « MODERN
OPERATING SYSTEMS » (4th Ed.) Pearson Education (2015).
- [3] William Stallings & Lawrie Brown, « Computer Security - Principles
and Practice » Pearson Education Limited (2018).